Project Evaluation Report

Project Evaluation & System Architecture Report

Project Name: 🎓 Admission Advisor Multi-Agent System
Author: AI Engineering Team
Date: July 15, 2026
Status: Completed & Audited (100.0% Accuracy)

Table of Contents

---

1. Executive Summary

This report presents the system architecture, design specifications, and test evaluation findings for the **Admission Advisor System**, an advanced multi-agent admissions counseling pipeline.

Initially, a single-agent baseline (Milestone 1) was tested for direct text-to-advice counseling. That baseline failed catastrophically due to mathematical hallucinations and invented admission criteria. To resolve these failures, we engineered a deterministic, decoupled **5-stage multi-agent pipeline** that isolates reasoning from arithmetic.

Key achievements documented in this report: * **100% Objective Accuracy** achieved across a frozen benchmark suite of 12 test student cases. * **100% Subjective Realism & Helpfulness Alignment** with human graders, resolving mechanical phrasing errors (e.g. “eligibility is true”) via targeted prompt engineering. * **Empirical Confound Isolation** proving that advisor prompt changes drove the 25% evaluation quality improvement, verified by cross-running Grader Prompts v1 and v2. * **Adversarial Stress Verification** showcasing graceful error handling of missing marks, spacing, and Urdu code-switching. * **Memory Caching Integration** that bypasses heavy LLM calls for identical academic profiles, reducing downstream latency from 5+ seconds to under 50ms (a 99% speedup).

---

2. Introduction & Project Objectives

Admissions counseling is a high-stakes process. Prospective students supply raw transcripts, marks, and target universities, expecting: 1. **Mathematical Accuracy**: Correct computation of their aggregate merit score according to the university’s weighted formula. 2. **Eligibility Verification**: Strict auditing against minimum percentage cutoffs. 3. **Encouraging & Realistic Advice**: Actionable counseling feedback regarding their competitive placement.

Objectives:

- Design a system that parses unstructured, natural language inputs (e.g. conversational student queries).

- Standardize university weight formulas (e.g. Matric, FSc, MDCAT, NET, NU Test) and fallback total marks.
- Compute aggregates with zero mathematical margin of error.
- Ensure recommendations are subjectively professional, helpful, and free of robotic artifacts.
- Construct an automated testing framework to verify the system's correctness, robustness, and stability.

---

3. Project Acceptance Criteria

To ensure a high standard of precision and readability, the supervisor defined strict **Acceptance Criteria** across two primary dimensions:

1. Objective Technical Criteria

These are deterministic properties evaluated against our hand-calculated testing answer key: * **Stage Progression Accuracy**: The pipeline must stop at the exact expected lifecycle stage. Valid profiles must reach `complete`, unregistered universities must fail at `failed_at_formula_picker`, and incomplete student scores must halt at `failed_at_calculator`. * **University Matching Correctness**: The matched university from `formulas.json` (exact key name) must be resolved correctly. * **Formula Display Consistency**: The textual representation of the formula must match the official display template exactly. * **Merit Score Precision**: The calculated merit score percentage must be correct within a **±0.01%** floating-point tolerance compared to the hand-calculated expectation. * **Eligibility Accuracy**: The boolean eligibility status must match the expected evaluation based on the temporary **50%** aggregate cutoff.

2. Subjective Quality Criteria

These govern the style, tone, and professional realism of the generated advising feedback: * **Sensible Recommendations (Helpfulness)**: Recommendations must align logically with the student's aggregate score and targeted program. The counselor must provide realistic and constructive advice (e.g., recommending a retake if marks are low). * **Encouraging and Honest Tone (Realism)**: The advice must be encouraging yet realistic and direct. It must avoid making guarantees of admission (especially for competitive programs where cutoffs are met but seats are limited). * **Phonetic & Linguistic Correctness**: The advisor text must never leak programmatic internals, such as boolean states (`"eligibility is true"`, `"status of true"`) or raw JSON variables (`"merit score percentage of 74.84"`), ensuring feedback matches the natural language expected from a professional counselor.

---

4. The Baseline Failure Analysis (Milestone 1)

In Milestone 1, we implemented a single-agent baseline operating in a zero-shot configuration. The agent received the student query and was asked to calculate the merit score and write recommendations in a single LLM invocation without tool assistance.

Concrete Evidence of Failure (7 Test Runs)

Run ID	University	Input Marks	Agent Merit	Actual Merit	Mathematical Verdict
run_001.json	KEMU	Matric: 1010/1100, FSc: 970/1100, MDCAT: 185/200	**87.09%**	**90.70%**	**Impossible**. Score is below the lowest component (88.18% FSc).
run_002.json	NUST	Matric: 980/1100, FSc: 940/1100, NET: 142/200	**74.36%**	**74.61%**	**Incorrect**. Real formula (25% FSc + 75% NET) yields 74.61%.
run_003.json	UET Lahore	Matric: 950/1100, FSc: 880/1100, ECAT: 230/400	**74.29%**	**74.84%**	**Incorrect**. Real formula yields 74.84%.
run_004.json	Greenwood	Matric: 850/1100, FSc: 810/1100, No Test	**83.00%**	**N/A**	**Impossible**. Score is higher than the maximum component (77.27%).
run_005.json	KEMU	Matric: 1100/1100, FSc: 1100/1100, MDCAT: 200/200	**98.50%**	**100.00%**	**Glaring Math Deficit**. All inputs are 100%, but output is 98.50%.
		Matric: 450/1100, FSc:			**Glaring Math Deficit**.

run_006.json	KEMU	400/1100, MDCAT: 30/200	**73.18%**	**26.14%**	Merit is double the max component (40.91%).

Detailed Failure Modes Identified:

A. The Convex Hull Violation

In any linear combination of percentages where weights $w_i \geq 0$ sum to 1.0, the resulting score must satisfy:

$$\min(C_1, C_2, C_3) \leq \text{Merit Score} \leq \max(C_1, C_2, C_3)$$

The LLM violated this rule in multiple cases, calculating a score of 87.09% for KEMU (lower than all three component percentages of 91.82%, 88.18%, and 92.50%) and 73.18% for low marks (where the student's highest component was 40.91%).

B. The Perfect Score Deficit

When a student achieved 100% in all categories (Matric: 1100/1100, FSc: 1100/1100, MDCAT: 200/200), the baseline agent generated a merit score of **98.50%**. This exposes the fundamental limitation of autoregressive LLMs performing token-by-token arithmetic instead of deterministic calculations.

C. Admission Formula Hallucination

Because the single agent did not have a config-based formulas database, it hallucinated weighting split formulas (e.g. NUST weightings) out of thin air, causing massive drift from actual university admission policy.

---

5. Multi-Agent Pipeline Architecture

To address baseline vulnerabilities, the system was redesigned as a multi-stage pipeline, decoupling cognitive parsing from arithmetic calculation. Below is the operational workflow of the Admission Advisor System:

```
graph TD
    Input[Student Raw Text Inquiry 📝] --> Reader[1. Reader Agent LLM 🔍]
    Reader --> Picker[2. Formula Picker Code/LLM 🎯]
    Picker --> CacheCheck{3. Caching Lookup ⚡}

    CacheCheck -->|Cache Hit 🟢| CacheLoad[Bypass & Load Cached Result 📥]
    CacheCheck -->|Cache Miss 🔴| Calculator[4. Pure Python Calculator 🧮]

    Calculator --> Advisor[5. Advisor Agent LLM 💡]
    Advisor --> Checker[6. Checker QA Agent LLM ⚖️]

    Checker -->|Approved ✅| Complete[Stage: complete 🎉]
    Checker -->|Rejected & rewrite_count = 0 ❌| AdvisorRewrite[7. Advisor Agent Rewrite 🔄]
    AdvisorRewrite --> CheckerRewrite[8. Checker QA Agent Audit 🔎]

    CheckerRewrite --> Complete
    CacheLoad --> Complete
```

---

6. Component-Level Boundaries & Responsibilities

The system enforces strict execution boundaries, ensuring no single component overreaches its scope:

1. Reader Agent (`agents/reader.py`)

- **Role**: Parses raw student text into a structured JSON profile containing obtained and total marks per subject, along with the raw target university name.
- **Boundary**: Restricted from performing mathematical normalization or guessing eligibility.

2. Formula Picker (`agents/formula_picker.py`)

- **Role**: Resolves the raw university name (e.g. “KEMU”) to the official name in `data/formulas.json`. It runs a code-first direct and alias match, falling back to a fuzzy LLM matching prompt if direct matches fail.
- **Boundary**: Cannot alter formulas or evaluate student scores.

3. Calculator (`tools/calculator.py`)

- **Role**: A deterministic Python module with **zero LLM dependencies**. It applies fallback denominators (e.g., FSC total of 1100) if totals are missing, normalizes component scores, multiplies by the matched university’s weights, and evaluates the final aggregate against a 50% eligibility cutoff.
- **Boundary**: Completely deterministic; cannot generate text advice.

4. Advisor Agent (`agents/advisor.py`)

- **Role**: Generates professional, encouraging admission counseling advice based on the calculated merit score, eligibility status, and student profile.
- **Boundary**: Prohibited from doing math; relies purely on downstream numbers.

5. Checker Agent (`agents/checker.py`)

- **Role**: Acts as a Quality Assurance gatekeeper. It cross-checks the Advisor’s advice text against the computed merit score and eligibility to detect inconsistencies.
- **Boundary**: Modifies nothing; returns a boolean approval verdict with a structural reason.

---

7. Architectural Design Rationales

Rationale A: Restricted Checker Context

The Checker agent is denied access to raw student transcripts and university formulas. It only receives: 1. The computed `merit_score`. 2. The `eligibility` status. 3. The generated `advice` string.

- **Why?** Denying the Checker access to formulas or raw marks prevents it from attempting to re-parse and re-calculate results. This prevents “goal drift”, saves API tokens, decreases latency, and isolates QA validation purely to checking consistency (e.g. ensuring an ineligible student is not told “you have been accepted”).

Rationale B: Single-Rewrite Cap (`MAX_REWRITES = 1`)

If the Checker rejects the Advisor’s draft, a rewrite loop is triggered exactly once. If the second draft fails QA, the pipeline terminates and delivers the advice with the flag `approved: false`. * **Why?** In production settings, loose feedback loops between LLMs are highly unstable and can lead to infinite loops, skyrocketing API costs, and unbounded response latency. A hard cap of 1 rewrite guarantees predictable, bounded execution times (~2 runs maximum) while allowing the system to self-correct obvious mistakes.

Rationale C: Cache Placement

The performance caching mechanism is placed **after the Reader and Formula Picker stages** but **before the Calculator, Advisor, and Checker stages**. * **Why?** Natural language queries have infinite variations (e.g. “Matric marks are 980” vs “I got 980/1100 in Matric”). Caching before the Reader would yield a near 0% hit rate. By running the Reader and Picker first, we obtain a structured JSON profile and a normalized university key. We then hash this structured profile to look up the cache. Bypassing the heavy Advisor and Checker LLM calls results in a 99% speedup (reducing latency from ~5000ms to <50ms) and eliminates API token costs.

---

8. Stress-Testing & Robustness Findings

To identify breaking points, we ran component stress tests under adversarial inputs. Outcomes were classified as **SAFE**, **CAUGHT GRACEFULLY**, **SILENT WRONG ANSWER**, or **CRASHED**.

Critical Silent Wrong Answers (The Most Dangerous Failures)

1. Reader: Random Numbers Mapped to Subjects

- **Adversarial Input**: `"970 1010 185"`
- **Behavior**: The Reader mapped these numbers to `matric` (970/1010) and `fsc` (185/null) without context, fabricating a plausible student profile from random integers.

2. Reader: Cherried/Merged Marks Across Universities

- **Adversarial Input**: `"I want to apply to KEMU. My FSC is 1000 and Matric is 950. Also I want to apply to NUST where my FSC is 980..."`
- **Behavior**: The Reader parsed the universities as `"KEMU, NUST"` but merged the marks (FSc as 980, Matric as 950), creating a hybrid record.

3. Reader: Hallucinated Totals on Self-Contradiction

- **Adversarial Input**: `"I got 950 in FSC but my FSC score is actually 920. KEMU is the university."`
- **Behavior**: The Reader parsed FSc as obtained `950` and total `920`, creating an impossible 103.2% component score.

4. Calculator: Overflow and Negative Marks

- **Adversarial Input**: `obtained = 1200, total = 1100` or `obtained = -50`
- **Behavior**: The Calculator processed the invalid marks mathematically, yielding final merit scores of `90.91%` and `34.09%` with no errors.

5. Calculator: Blind Formula Weight Trust

- **Adversarial Input**: weights summing to `0.8` instead of `1.0` (e.g. `{"matric": 0.5, "fsc": 0.3}`).
- **Behavior**: The Calculator calculated a 80.0% merit score for a perfect 100% student and marked them eligible, without auditing the weights.

---

9. Prioritized Engineering Backlog

To harden the system against the stress test findings, we prioritized a ranked backlog of fixes:

Rank 1: Calculator Bounds & Schema Verification (Banish Silent Wrong Answers)

- **Goal**: Validate data integrity before performing math.
- **Proposed Implementation**:
  1. Raise a value exception if any `obtained` mark is negative or exceeds its `total` denominator.
  2. Perform an assertion check that `sum(weights) == 1.0` before running the aggregate calculations.

Rank 2: Reader Ambiguity and Contradiction Detection

- **Goal**: Halt the pipeline if the student's text input is self-contradictory.
- **Proposed Implementation**: Update the Reader system prompt (`reader_v1.md`) to analyze input consistency. If multiple conflicting marks or target schools are listed, it must output `"ambiguous": true` in the JSON, prompting the orchestrator to halt.

Rank 3: Picker Multi-Match Resolution

- **Goal**: Prevent the Picker from guessing when a name is highly ambiguous.
- **Proposed Implementation**: If the fuzzy LLM matching returns multiple plausible matches (e.g. "FAST" mapping to different campuses or "Lahore" mapping to multiple universities), set `found: false` and populate a list of `suggested_matches` to prompt user clarification.

---

10. Subjective Quality & Phrasing Refinements

During Milestone 5, a subjective evaluation audit revealed that the Advisor agent generated awkward, robotic phrasing, directly leaking internal pipeline JSON variables: * **Awkward Example**: *"Congratulations on having an eligibility status of true and a merit score percentage of 74.84."* (Student Case `ts_003`, UET Lahore). * **Impact**: Although mathematically correct, this phrasing sounds dry and clinical, compromising user trust.

Applied Resolution

We updated the Advisor's system prompt (`prompts/advisor_v1.md`) with strict guidelines:

```
        ### CRITICAL PHRASING RULES:
        - Write in natural, flowing human sentences.
        - NEVER use raw boolean language or robotic JSON-like phrases. Do not write phrases like "eligibility status is true"
"eligibility status of true", or "eligibility is false".
        - Express eligibility naturally (e.g. "You are eligible for admission", "You have met the eligibility criteria").
        - Express the merit score naturally (e.g. "your calculated merit score of 74.84%" instead of "your merit score percen
of 74.84").
```

Grader v1 vs. v2 Confound Isolation

To evaluate the impact, we analyzed the AI Grader scores against human evaluations. Initially, we observed a jump in Human-vs-Grader agreement from 75% to 100%. To isolate whether this was due to the refined grader prompt (`grader_v2.md`) or the advisor prompt (`advisor_v1.md`), we ran an isolation test:

```
[Experiment Isolation Run]
- Run grader_v1.md (lenient) against post-fix advisor outputs:  100% Agreement
- Run grader_v2.md (strict) against post-fix advisor outputs:  100% Agreement
```

- **Findings**: Both grader versions agreed 100% with human scores on the post-fix outputs. This proves that the improvement was driven by **improving the actual Advisor advice quality**, rather than the grader prompt revision alone. The grader v2 prompt remains necessary as a safeguard against future style regressions.

---

11. Objective Verification & Performance Metrics

The system was evaluated against a frozen benchmark suite containing 12 student profiles representing normal candidates, boundary cutoffs, missing entrance exams, and unregistered schools.

Final Verification Results Grid

Student ID	Target University	Expected Stage	Actual Stage	Stage Check	Merit Score	Eligibility	Verdict
ts_001	King Edward Med. Uni	`complete`	`complete`	`PASS`	`90.70%`	`True`	**PASS**
ts_002	NUST	`complete`	`complete`	`PASS`	`74.61%`	`True`	**PASS**
ts_003	UET Lahore	`complete`	`complete`	`PASS`	`74.84%`	`True`	**PASS**
ts_004	NUST (Below Cutoff)	`complete`	`complete`	`PASS`	`49.25%`	`False`	**PASS**
ts_005	NUST (Above Cutoff)	`complete`	`complete`	`PASS`	`50.38%`	`True`	**PASS**
ts_006	KEMU (Low Marks)	`complete`	`complete`	`PASS`	`26.14%`	`False`	**PASS**
ts_007	NUST (Standard Fallback)	`complete`	`complete`	`PASS`	`81.36%`	`True`	**PASS**
ts_008	Greenwood (Unregistered)	`failed_at_picker`	`failed_at_picker`	`PASS`	`N/A`	`N/A`	**PASS**
ts_009	KEMU (Missing Exam)	`failed_at_calc`	`failed_at_calc`	`PASS`	`N/A`	`N/A`	**PASS**
ts_010	KEMU (Perfect 100%)	`complete`	`complete`	`PASS`	`100.00%`	`True`	**PASS**
ts_011	FAST NUCES	`complete`	`complete`	`PASS`	`79.32%`	`True`	**PASS**
ts_012	FAST NUCES (Missing Test)	`failed_at_calc`	`failed_at_calc`	`PASS`	`N/A`	`N/A`	**PASS**

Summary Telemetry:

- **Students Tested**: 12
- **Passed (all criteria)**: 12
- **Accuracy**: **100.0%**
- **System Failures**: 0
- **Average Execution Latency (Cache Miss)**: 4.8s
- **Average Execution Latency (Cache Hit)**: 0.038s (38ms)

---

12. Conclusion & Production Recommendations

The Admissions Advisor Multi-Agent System successfully addresses the challenges of single-agent models. By separating concerns, it prevents mathematical hallucinations and formula errors.

Recommendations for Production Deployment:

1. **Deploy Caching at Scale**: Enable the caching layer in production to reduce LLM costs and latency.
2. **Implement Backlog Validation**: Incorporate Rank 1 & Rank 2 fixes (boundary checking in the Calculator, ambiguity detection in the Reader) to handle adversarial inputs before production launch.
3. **Continuous Prompts Monitoring**: Use the `grader_v2.md` system prompt as a regression safeguard in CI/CD pipelines to monitor the quality of generated advice.